

An Experimental Study of Parallel Sorting by Regular Sampling on Shared-Memory Systems

Saumya Patel

Student Number: 1812526

CCID: snp2

1 Introduction

This report describes the implementation and experimental evaluation of Parallel Sorting by Regular Sampling (PSRS) in a shared-memory environment using POSIX threads. The implementation was evaluated on randomly generated integer arrays ranging from one million to sixty-four million keys, with worker threads varied from 2 to 256. All experiments were compiled with aggressive optimization flags (`-O3 -march=native`) to ensure that the baseline performance reflects realistic production code. Rather than focusing solely on theoretical asymptotic bounds, this work examines how the algorithm behaves in practice on a modern multicore system, specifically the Apple M4 architecture.

In addition to measuring overall execution time and speedup, the experiments record the execution time of each phase of the algorithm. This phase-level instrumentation makes it possible to observe how individual components scale and how their relative costs change as the level of parallelism increases. The results demonstrate that PSRS achieves a peak speedup of $4.67\times$ at 32 threads, after which scalability is limited by hardware constraints. Phase-level measurements reveal that while the local sorting phase scales linearly, the later stages involving synchronization and data movement increasingly dominate execution time, creating a "memory wall" that prevents linear scaling at high thread counts.

2 Implementation Details

This section describes the practical design choices made while implementing PSRS. The focus is on how the abstract algorithm was realized in code to maximize efficiency on shared-memory hardware.

2.1 Thread Model and Phase Structure

The implementation follows a single-program multiple-data (SPMD) execution model, where a fixed number of worker threads equal to the requested processor count execute the same code on disjoint portions of the input array. Threads are created once at the start of execution and persist across all phases of the algorithm.

Phase boundaries are enforced using barrier synchronization, ensuring that all threads complete a phase before proceeding to the next one. This monolithic thread pool approach avoids the overhead of repeated thread creation and destruction (`fork/join`), which can significantly impact performance at higher thread counts where the work-per-thread is small.

2.2 Synchronization and Phase Boundaries

Synchronization is strictly limited to phase boundaries. Within each phase, threads operate independently on private or disjoint memory regions without fine-grained locking. Barriers are used to ensure that all threads complete a phase before shared data structures—such as the array of pivots or the count of elements per bucket—are accessed by subsequent phases.

Although barrier synchronization introduces overhead (latency increases as $O(\log p)$ or $O(p)$ depending on the barrier implementation), limiting synchronization to a small number of well-defined points helps contain its impact. However, at high thread counts ($p > 64$), the cost of threads "checking in" to the barrier begins to constitute a measurable fraction of the runtime.